claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a9ecb0b2f9a51ffea.jsonl`
- SHA-256: `097dcc4643e6ea3a3e4dd7a091480f055eecbc73f4cc2201530cb6f4326d2bfc`
- Source modified: `2026-07-08T06:15:16+00:00`
- Imported at: `2026-07-08T15:59:28+00:00`
- project: `wf_5f13fc79-bad`
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`

Transcript

1. user (2026-07-08T06:12:31.308Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to
an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).
The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):
- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical
  to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the
  SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config
  + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed
  once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing,
  never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a
wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT
report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is_real=false if you cannot concretely confirm the failure. Skeptic #1.

CLAIM (medium/robustness) backend/orchestration.py:689

summary: When validation is delegated to the L4 worker
(phase_placement.validation=cloud_run_l4), a worker validation REJECTION surfaces to the user as
an opaque "cloud job failed (returncode 2)." with no reason, silently dropping the validation
feedback the in-process path provides.
failure scenario: Config: deployment.phase_placement.validation=cloud_run_l4 with
segment=cloud_run_l4 (or compute_target=cloud_run). A user uploads a too-blurry / not-badminton
clip. _execute_processing sets validate_on_control_plane=False (orch:865-868), so the control-
plane skips its validators and dispatches with skip_validation=worker_skips_validation=False
(orch:666-674). The worker's blur/relevance validator rejects the clip: cmd_infer hits `if
failures: return _fail(2)` (worker/__main__.py:333-338), which writes a done-marker via
_write_done_marker containing ONLY {video_id, returncode:2, segment_count:0, status:"failed"} ?
NO message/validator-reason field (worker/__main__.py:228-238). run_infer returns
ComputeResult(returncode=2); _maybe_dispatch_remote line 688-689 then returns _failed(f"cloud
job failed (returncode {result.returncode}).") ? it never reads result.report.get('message')
(unlike the compile path at orch:1393). The user sees "cloud job failed (returncode 2)." instead
of the specific 'video too blurry / not badminton' message the in-process control-plane path
returns. This failure mode is newly reachable in #521 (pre-#521 skip_validation was hardcoded
True so the worker never validated) and is untested ? the new tests
(test_validation_delegated_to_worker_when_placed_on_l4 / _not_delegated_...) only cover the PASS
path and never assert the worker-rejection message.

2. assistant (2026-07-08T06:12:37.796Z)

I'll investigate this claim by reading the actual code. Let me start with the key files.

3. user (2026-07-08T06:12:38.831Z)

```
620                 False,
621             )
622
623         notify("dispatching (cloud_run)")
624         logger.info(
625             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
626             req.video_path,
627             out_root,
628         )
629         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
630         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
631         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
632         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
633         overrides = []
634         idx_cfg = getattr(cfg, "indexing", None)
635         sk = getattr(idx_cfg, "skip_ai_handoff", None)
636         if sk is not None:
637             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
638         # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
639         # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
640         # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
641         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
642         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
643         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
644         wasb_cfg = getattr(idx_cfg, "wasb", None)
645         if wasb_cfg is not None:
646             decode_backend = getattr(wasb_cfg, "decode_backend", None)
647             if decode_backend:
```

```
                overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
            batch_size = getattr(wasb_cfg, "batch_size", None)
            if batch_size:
                overrides.append(f"indexing.wasb.batch_size={batch_size}")
            prefetch = getattr(wasb_cfg, "prefetch_batches", None)
            if prefetch:
                overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
        # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
        # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
        # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
        # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
        # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
        # then worker-rejected at its default 20.0 after a successful GPU dispatch).
        # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the
control-plane is
        # the authoritative gate ? it already validated with its resolved config (only a
PASS reaches
        # here), so the worker SKIPS its redundant re-validation (#513). If validation is
placed on the
        # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators ?
see
        # _execute_processing), so the worker must NOT skip: it becomes the gate.
        worker_skips_validation = (
            resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
        )
        spec = ComputeJobSpec(
            video_uri=req.video_path,
            out_uri=out_root,
            segmenter=seg_name,
            config_overrides=tuple(overrides),
            skip_validation=worker_skips_validation,
        )
        try:
            # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
            # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
            # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
            # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
            result = compute.run_infer(
                spec,
                on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
            )
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
            return _failed(f"cloud dispatch error: {e}", True)
        if result.returncode != 0:
            return _failed(f"cloud job failed (returncode {result.returncode}).", True)
        if result.video_id and result.video_id != video_id:
            # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
            # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
            logger.warning(
                "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
                "and the worker hash; ingesting under the local id.",
                result.video_id,
                video_id,
```

```
698             )
699
700         notify("ingesting (cloud_run)")
701         storage_dict = (
702             storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
703             if hasattr(storage_cfg, "model_dump")
704             else {}
705         )
706         try:
707             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
708         except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
709             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
710             return _failed(
711                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
712                 True,
713             )
714
715         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
716         segments = [
717             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
718         ]
719         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
720         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
721         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
722         provenance = {
723             "path": "cloud_run",
724             "requested": req.compute,
725             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
726             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
727             "execution": result.execution,
728             "outputs_uri": result.outputs_uri,
729         }
730         logger.info(
731             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
732             result.execution,
733             provenance["job"],
734             provenance["region"],
735             result.outputs_uri,
736             video_id,
737         )
738         return (
739             {
```

4. user (2026-07-08T06:12:39.470Z)

```
200                 "[worker] remote execution terminated without a completion marker: %s", e
201             )
202             return 5
203         except PolicyTimeout as e:
204             logger.warning(
205                 "[worker] remote compute did not complete (no marker within budget): %s", e
206             )
207             return 4
208         logger.info(
209             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210             result.video_id,
211             result.returncode,
212             result.outputs_uri,
213         )
```

```
        return result.returncode


    def _write_done_marker(
        done_uri: str,
        video_id: str,
        returncode: int,
        segment_count: int,
        status: str,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(
            args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
        )
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
```

```
271         video_id = compute_video_id(local_video)
272
273         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282         if getattr(args, "skip_validation", False):
283             print(
284                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285                 "gate and already validated this input."
286             )
287             val_results, val_context = [], {}
288         else:
289             val_results, val_context = validator_registry.run_all_with_context(
290                 local_video, config
291             )
292         failures = [r for r in val_results if not r.passed]
293         db = Database(os.path.join(scratch, config.output.db_name))
294         db.add_video(
295             video_id,
296             local_video,
297             "failed" if failures else "processed",
298             [r.model_dump() for r in val_results],
299         )
300
301         def _fail(rc: int) -> int:
302             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
303             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
304             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
305             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
306             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
307             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
308             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
309             try:
310                 _serialize_and_push_outputs(
311                     scratch,
312                     args.out_uri,
313                     video_id,
314                     [],
315                     "failed",
316                     storage_cfg,
317                     str(getattr(args, "video_uri", "") or ""),
318                 )
319             except Exception as e:  # never mask the real failure on an artifact-push hiccup
```

```
320                 logger.warning("[worker] could not push failure artifacts: %s", e)
321             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
322             if getattr(args, "done_uri", None):
323                 _write_done_marker(
324                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
325                 )
326             return rc
327 
328         segments: List[dict] = []
329         segmenter_actual = None
330         segmentation_ran = False
331         status = "failed"
332         try:
333             if failures:
334                 print(
335                     "[worker] validation FAILED: "
336                     + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
337                 )
338                 return _fail(2)
339             seg_name = args.segmenter or config.indexing.default_segmenter
340             segmenter_cls = segmenter_registry.get(seg_name)
341             if not segmenter_cls:
342                 print(
343                     f"[worker] unknown segmenter: {seg_name!r} "
344                     f"(have {list(segmenter_registry.list_available())})"
345                 )
346                 return _fail(2)
347             segmenter_actual = seg_name
348             result = segmenter_cls(db, config).process_video(
349                 video_id, local_video, max_frames=args.max_frames
350             )
351             segmentation_ran = True
352             if isinstance(result, Failure):
353                 print(f"[worker] segmenter FAILED: {result.message}")
354                 return _fail(3)
355             segments = result.value if hasattr(result, "value") else result
356             status = "success"
357             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
```

5. assistant (2026-07-08T06:13:15.525Z)

Let me read more of the orchestration file, including the validate_on_control_plane logic, the compile path, and the config guard.

6. user (2026-07-08T06:13:18.116Z)

```
820         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
821         compute_actual: Optional[str] = None
822         response: Optional[Dict[str, Any]] = None
823         try:
824             if (
825                 existing_video
826                 and existing_video.get("status") == "processed"
827                 and not req.force
828             ):
829                 cached_segments = db.query_play_segments(video_id, {})
830                 if cached_segments:
831                     logger.info(
832                         "[API] Reusing cached segmentation for video_id=%s; force=true
```

```
bypasses cache.",
                        video_id,
                    )
                    response = _cached_process_payload(
                        video_id,
                        existing_video.get("validation_results", []),
                        cached_segments,
                        getattr(req, "compute", None),
                    )
                    return response

            # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
            #    UNLESS this exact content already has a cached verdict under the current
validator config.
            notify("validating")
            # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
            # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
            # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
            # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
            # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
            # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
            # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
            # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
            # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
            # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
            # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
            #
            # #521 phase_placement.validation: when validation is placed on the L4 AND
detection will
            # actually dispatch there, the control-plane DELEGATES validation to the worker
? it does NOT
            # run the validator suite here (the whole point is to move the CPU-bound
validation OFF the
            # 2-vCPU control-plane). The worker then validates (skip_validation=False) and
rejects a bad
            # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed job.
When detection
            # is NOT on the cloud (in-process), the control-plane always validates ? there
is no worker to
            # delegate to (the config guard makes validation=cloud_run_l4 require a cloud
segment).
            validate_on_control_plane = (
                resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
                or not _will_detect_on_cloud(config, getattr(req, "compute", None))
            )
            if not validate_on_control_plane:
                logger.info(
                    "[API] validation delegated to the L4 worker "
                    "(deployment.phase_placement.validation=cloud_run_l4) ? the control-
plane skips its "
                    "validator suite; the worker is the gate for %s.",
                    req.video_path,
                )
```

```
            val_fingerprint = (
                registry.config_fingerprint(config) if validate_on_control_plane else ""
            )
            # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
            # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
            # failing the job. The cache is a pure speedup ? never a new failure mode.
            reused_validation = False
            if validate_on_control_plane and not req.force:
                try:
                    cached = db.get_validation_cache(video_id)
                    if cached is not None and cached.get("fingerprint") == val_fingerprint:
                        # Replay the stored ValidationResult list; val_context stays {} (no
fresh
                        # ffprobe_meta ? the report's media block degrades gracefully to
size-only, the
                        # deliberate cost of skipping the decode). Segmentation reads
neither of these.
                        val_results = [
                            ValidationResult(**r) for r in cached.get("results", [])
                        ]
                        reused_validation = True
                        logger.info(
                            "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
                            "validator config unchanged ? skipping the validator suite. "
                            "force=true re-validates.",
                            cached.get("passed"),
                            video_id,
                        )
                except Exception:  # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
                    reused_validation = False
                    logger.warning(
                        "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
                        video_id,
                        exc_info=True,
                    )
            if validate_on_control_plane and not reused_validation:
                logger.info(f"Running validations for {req.video_path}...")
                val_results, val_context = registry.run_all_with_context(local_video,
config)
            failures = [r for r in val_results if not r.passed]
            if validate_on_control_plane and not reused_validation:
                # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
                # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
                db.set_validation_cache(
                    video_id,
                    val_fingerprint,
                    not failures,
                    [r.model_dump() for r in val_results],
                )

            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db.add_video(
                    video_id,
                    req.video_path,
                    "failed",
                    [r.model_dump() for r in val_results],
```

```
930                 )
931                 response = {
932                     "video_id": video_id,
933                     "status": "failed",
934                     "message": _validation_failure_message(failures),
935                     "results": [r.model_dump() for r in val_results],
936                 }
937                 return response
938
939             # 2. Run segmenter & indexer
```

7. user (2026-07-08T06:13:18.638Z)

```
1340        notify,
1341    ) -> Dict[str, Any]:
1342        """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
1343        control-plane owns the DB, so it RESOLVES the reel's segments here
(``_resolve_compile``), stages a
1344        small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
1345        to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
1346        done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
1347        worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
1348        ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
1349        from backend.compute import CompileJobSpec, get_compute_backend
1350
1351        notify("resolving")
1352        try:
1353            video, kept, out_name = _resolve_compile(
1354                db, cfg, video_id, segment_ids, output_filename
1355            )
1356        except _CompileError as e:
1357            return {"status": "failed", "message": e.detail, "video_id": video_id}
1358
1359        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1360        storage_cfg = getattr(cfg, "storage", None)
1361        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
1362        compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
1363        if compute is None:
1364            # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
1365            # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ?
a slow reel
1366            # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
1367            logger.warning(
1368                "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
1369                "stitching in-process on the control-plane instead.",
1370                out_root,
1371            )
1372            time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1373            return _stitch_reel(
1374                cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
1375            )
1376
1377        # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe
on the wire);
1378        # the stitching config is the control-plane's RESOLVED one, with the encoder swapped
to the L4's
```

```
1379        # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes
the reel.
1380        stitch_dump = stitching.model_dump()
1381        stitch_dump["encoder"] = getattr(
1382            getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
1383        )
1384        spec_payload = {
1385            "video_id": video_id,
1386            "video_uri": video["filepath"],
1387            "output_name": out_name,
1388            "locale": locale,
1389            "time_pairs": [
1390                [float(s["start_time"]), float(s["end_time"])] for s in kept
1391            ],
1392            "stitching": stitch_dump,
1393        }
1394        token = uuid.uuid4().hex
1395        spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
1396        try:
1397            notify("staging compile spec")
1398            with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
1399                local_spec = os.path.join(td, "spec.json")
1400                with open(local_spec, "w", encoding="utf-8") as f:
1401                    json.dump(spec_payload, f)
1402                storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
1403        except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
1404            logger.error(
1405                "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
1406            )
1407            return {
1408                "status": "failed",
1409                "message": f"cloud compile: could not stage the compile spec: {e}",
1410                "video_id": video_id,
1411            }
1412
1413        # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
1414        # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
1415        overrides = [f"storage.output_root={out_root}"]
1416        backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
1417        if backend_name:
1418            overrides.append(f"storage.backend={backend_name}")
1419        spec = CompileJobSpec(
```

8. assistant (2026-07-08T06:13:23.319Z)

Let me read the rest of the compile path to find where it reads `result.report.get('message')`.

9. user (2026-07-08T06:13:24.693Z)

```
1419        spec = CompileJobSpec(
1420            out_uri=out_root,
1421            spec_uri=spec_uri,
1422            video_id=video_id,
1423            output_name=out_name,
1424            config_overrides=tuple(overrides),
1425        )
1426
1427        notify("dispatching stitch (cloud_run)")
1428        logger.info(
1429            "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
```

```
            video_id,
            out_root,
            len(kept),
        )
        try:
            result = compute.run_compile(
                spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
            )
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
            return {
                "status": "failed",
                "message": f"cloud compile dispatch error: {e}",
                "video_id": video_id,
            }
        if result.returncode != 0:
            msg = str(
                result.report.get("message")
                or f"cloud compile job failed (returncode {result.returncode})."
            )
            return {"status": "failed", "message": msg, "video_id": video_id}

        download_url = str(
            result.report.get("download_url")
            or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
        )
        logger.info(
            "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
            result.execution,
            result.outputs_uri,
            video_id,
        )
        notify("finished")
        return {
            "status": "success",
            "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
            "download_url": download_url,
            "video_id": video_id,
            "compute": {
                "path": "cloud_run",
                "execution": result.execution,
                "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
                "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
            },
        }


    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
        """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
        the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
        result for an EXPECTED problem (missing video / no segments / a stitch error) so the
SPA always gets
        a clean verdict instead of a worker-crash error.

        #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``)
and cloud
        dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
        (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
        2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process
here, unchanged."""
        notify = progress or (lambda stage: None)
```

```
1488        params = json.loads(job.get("params") or "{}")
1489        video_id = job.get("video_id") or ""
1490        segment_ids = params.get("segment_ids") or []
1491        locale = params.get("locale")
1492        output_filename = params.get("output_filename", "")
1493
1494        if (
1495            resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
1496            and _cloud_available(cfg)
1497        ):
1498            return _dispatch_compile_remote(
1499                cfg, db, video_id, segment_ids, output_filename, locale, notify
1500            )
1501
1502        notify("resolving")
1503        try:
1504            video, kept, out_name = _resolve_compile(
1505                db, cfg, video_id, segment_ids, output_filename
1506            )
1507        except _CompileError as e:
1508            return {"status": "failed", "message": e.detail, "video_id": video_id}
1509
1510        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1511        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1512        return _stitch_reel(
1513            cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
1514        )
1515
```

10. assistant (2026-07-08T06:14:00.976Z)

Let me check the compile worker's marker-writing to understand the asymmetry, and verify what `_serialize_and_push_outputs` includes on failure.

11. user (2026-07-08T06:14:03.142Z)

```
217:def _write_done_marker(
323:            _write_done_marker(
336:                + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
353:            print(f"[worker] segmenter FAILED: {result.message}")
416:        _write_done_marker(
436:def _write_compile_marker(
444:    """#521: write the compile done-marker (rc + status + the reel's download_url/reel_uri)
so the
451:            "download_url": result.get("download_url", ""),
452:            "reel_uri": result.get("reel_uri", ""),
453:            "message": result.get("message", ""),
464:def cmd_compile(args: argparse.Namespace) -> int:
516:        print(f"[worker] compile FAILED: {result.get('message')}")
520:        _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
884:        help="storage URI for the compile completion MARKER (rc + download_url), so a remote
"
964:        format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
```

12. assistant (2026-07-08T06:14:41.660Z)

I have enough to adjudicate. Let me confirm the two key facts by reading the exact marker-writing code once more.

13. user (2026-07-08T06:14:42.529Z)

```
436     def _write_compile_marker(
437         done_uri: str,
```

```
438         result: dict,
439         video_id: str,
440         returncode: int,
441         storage_cfg: dict,
442         scratch: str,
443     ) -> None:
444         """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
445         dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
446         try:
447             marker = {
448                 "video_id": video_id,
449                 "returncode": returncode,
450                 "status": result.get("status", "failed"),
451                 "download_url": result.get("download_url", ""),
452                 "reel_uri": result.get("reel_uri", ""),
453                 "message": result.get("message", ""),
454             }
455             local = os.path.join(scratch, "_compile_done.json")
456             with open(local, "w", encoding="utf-8") as f:
457                 json.dump(marker, f)
458             storage.put(local, done_uri, config=storage_cfg)
459             logger.info("[worker] wrote compile marker -> %s", done_uri)
460         except Exception as e:  # never fail a good stitch because the marker push hiccuped
461             logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)
462
463
464     def cmd_compile(args: argparse.Namespace) -> int:
465         """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
stitch is the
```

14. assistant (2026-07-08T06:15:08.409Z)

I've now confirmed all the load-bearing facts and can adjudicate.

Verified facts (all accurate as the reviewer states):
- `_write_done_marker` (worker/__main__.py:229-234) for the detect/infer path writes ONLY `{video_id, returncode, segment_count, status}` ? no `message`.
- `_write_compile_marker` (worker/__main__.py:447-454) DOES write a `message` field ? hence the compile path at orch:1445-1450 reads `result.report.get("message")`.
- The infer dispatch at orch:688-689 returns `_failed(f"cloud job failed (returncode {result.returncode}).")` and never reads `result.report`.
- The in-process control-plane path (orch:931-936) returns a specific `_validation_failure_message(failures)`.
- The delegation is reachable: `validate_on_control_plane` (orch:865-868) is False when `validation=cloud_run_l4` AND detection dispatches on cloud, and `worker_skips_validation` (orch:666-668) is then False, so the worker validates and can `_fail(2)`.

So the factual mechanics are correct. But this does not clear the task's defect bar, and I judge it NOT a reportable defect:

1. **Correct verdict, no wrong result / crash / silent data loss.** A blurry/not-badminton clip IS correctly rejected ? the job returns `status: failed`. The only difference is message granularity ("returncode 2" vs a specific reason). No bad data is ingested (`_finalize_reingest` runs only on rc==0), nothing crashes, and the failure is surfaced loudly, not silently.

2. **Not truly "dropped" / silent.** On rejection the worker's `_fail` (worker/__main__.py:301-326) calls `_serialize_and_push_outputs(... "failed" ...)`, pushing a `status=failed` report.json (with validator results persisted in the worker DB) to the bucket. The reason is retained for diagnostics; it's just not echoed into the job's top-line message.

3. **Deliberate, documented behavior on an opt-in, non-default path.** orch:858-864 explicitly states the worker "rejects a bad clip via a non-zero rc, which `_maybe_dispatch_remote` surfaces

as a failed job." DEFAULT-OFF is preserved: the default `control_plane` validation still returns the specific message (orch:934). Only the explicitly-configured `validation=cloud_run_l4` delegation yields the coarse message.

4. **The reviewer's own label is "robustness."** The task instructs: report only wrong-result/crash/silent-data-loss/missing-coverage-of-a-real-failure-mode, and explicitly to NOT report style or speculative concerns. Message-granularity on a non-default path is a UX/robustness degradation. "Missing test coverage" here covers a path that behaves *correctly* (just coarsely), so it doesn't elevate this to a defect.

This is a genuine behavioral asymmetry worth a follow-up nicety, but not a defect that produces a wrong result, crash, or silent data loss. Per the instruction to default to is_real=false absent a concretely confirmed failure, I refute it.

15. user (2026-07-08T06:15:15.933Z)

Structured output provided successfully